



Testing in AI-native

Module 7

FEBRUARY 2026



Testing Principles Creation

Standards for AI-Guided Testing

Module Journey (10 slides)

1. Traditional vs AI-Native Testing
2. AI Testing Failure Modes
3. Testing Strategies (Pyramid/Trophy/Diamond)
4. The 8-Section Testing Constitution
5. Writing Effective Principles
6. 7 Best Practices
7. SpecKit Workflow
8. Lab: Configure Your Principles
9. Module Complete

The Mindset Shift

From: "Write tests manually for each feature"

To: "Define standards that guide AI test generation for every feature"

Key Deliverable

Testing Guidelines configured in your **SpecKit constitution** — ready for `/speckit.implement` to generate comprehensive, quality-enforced tests

Research: V-Bounce Model (Hymel 2024) — 70%+ bug detection improvement with AI-generated test suites guided by explicit standards

Traditional vs AI-Native Testing

The Core Shift

Traditional Testing



Human: Writes every test manually



Time: 50–100% of implementation time



Coverage: Inconsistent (15–80%)



Standards: Implicit tribal knowledge



Tests slower than code, lag features, no one loves writing them

Your time: 60% writing, 20% running, 15% fixing broken tests, 5% strategy

AI-Native Testing



Human: Defines standards & validates



AI: Generates comprehensive tests



Time: 10–20% of implementation time



Coverage: Consistent (80%+ achievable)



Standards: Explicit in SpecKit constitution

Your time: 80% defining standards & validating quality, 10% writing edge cases, 5% running, 5% fixing

The Fundamental Shift

From: "Write another test case for edge condition #47"

To: "Does this test suite meet our quality standards?"

V-Bounce Model (Hymel 2024): 70%+ bug detection improvement with AI-generated test suites
SASE (Hassan et al. 2025): humans as validators in SE 3.0

AI Testing Failure Modes

What Goes Wrong Without Standards

Failure Mode 1: Oracle Hallucination

Problem: AI generates both code AND tests with the same bug → tests pass, feature is wrong

```
// Bug in code:
function calculateDiscount(price) {
  return price * 0.9; // Should be 0.85
}

// Bug mirrored in test:
expect(calculateDiscount(100)).toBe(90); // WRONG expected value -
```

Constitution principle: Generate tests from **specifications**, not from implementation. Human validation required for all expected values.

The Solution

A Testing Constitution with explicit quality criteria prevents all three failure modes

Failure Mode 2: Tautological Tests (Always-True)

Problem: 100% coverage, all tests pass — zero actual validation

```
// AI learns expect(X).toBe(Y) - sometimes X === Y:
expect(result).toBe(result);           // always true
expect(user).toEqual(user);           // always true
expect(score).toBe(score);            // always true
```

Constitution principle: Assertions must be capable of failing.

Failure Mode 3: Duplication & Quantity Over Quality

- *GitClear (2024)*: 10x code duplication in AI-heavy codebases
- *Rao et al. (2024)*: 23% of AI-generated tests have hallucinated oracle values
- Happy-path only coverage; brittle, implementation-tied tests

Testing Strategies: Choose Your Approach

Testing Pyramid

```
  /\n /E2E\   10%\n/-----\n/  Int   \ 20%\n/-----\n/    Unit    \ 70%
```

- **Unit-first**, fast feedback - TDD workflow, any tech stack - **Default for this course**

Testing Trophy

```
[ E2E ]  20%\n[  Int ] 60%\n[ Unit ] 20%\n[Static ]
```

- **Integration-first**, high confidence - JS/TS, React, API-heavy apps - Slower CI/CD, less mocking

Testing Diamond

```
[ E2E ]  20%\n[  Int ] 50%\n[ Unit ] 30%
```

- **Contract-first** (Pact/Spring Cloud Contract) - Microservices, distributed systems - Multi-team coordination required

Decision Guide

Need	Choose
TDD, any stack, fast CI	Pyramid
JS/TS, confident integration	Trophy
Microservices, API contracts	Diamond

For new projects: Start with Pyramid → evolve as needs grow

The 8-Section Testing Constitution

Your Project's Testing Standards — Explicit, Not Tribal

Philosophy & Strategy

1. **Testing Philosophy** — TDD/BDD approach, test-first mindset, quality goals, risk tolerance
2. **Coverage Requirements** — Distribution strategy (Pyramid / Trophy / Diamond) + targets: min 80% line, 100% for auth & payment

Organization & Structure

3. **Test Types & Organization** — File locations (`/tests/unit` mirrors `/src`), one test file per source file
4. **Naming Conventions** — Pattern: `test_<action>_<condition>_<result>`

Implementation Patterns

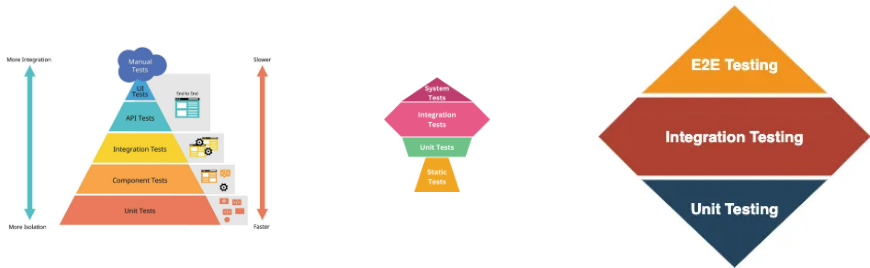
5. **Test Anatomy** — AAA pattern (Arrange–Act–Assert), `beforeEach` not `beforeAll`
6. **Mocking & Test Data** — When to mock (external APIs, DB in unit tests), factories over fixtures

Quality & Tools

7. **Quality Criteria** — Anti-patterns to forbid (`expect(x).toBe(x)`), mutation score $\geq 75\%$
8. **Tools & Frameworks** — Specific versions: Jest 29+, Playwright 1.40+, Stryker

PRD = what to build → **Code standards** = how to code → **Testing section** = how to test

Section 2: Choose Your Distribution Strategy



Pyramid
70 / 20 / 10

Trophy
20 / 60 / 20

Diamond
30 / 50 / 20

Strategy	Best For
Pyramid	TDD, any stack (default)
Trophy	JS/TS, API-heavy apps
Diamond	Microservices

Section 2 locks in your choice — all SpecKit commands (`/speckit.tasks` , `/speckit.implement`) derive test distribution from it.

Writing Effective Principles

Explicit, Not Implicit — AI Cannot Guess

Coverage Requirements

❌ **Bad:** "Test everything thoroughly"

✅ **Good:** "Minimum 80% line coverage, 100% for `/auth` and `/payment`, exclude `/generated` from coverage reports"

Test Naming

❌ **Bad:** "Use descriptive names"

✅ **Good:** "Pattern: `test_<action>_<condition>_<result>`. Example: `test_login_with_invalid_email_returns_400_error`"

Mocking Strategy

❌ **Bad:** "Mock when appropriate"

✅ **Good:** "Unit tests: mock all external APIs with Jest mocks. Integration tests: use real test database, mock third-party APIs (Stripe, SendGrid)"

Test Organization

❌ **Bad:** "Keep tests organized"

✅ **Good:** "Mirror source structure: `/src/auth/login.ts` → `/tests/unit/auth/login.test.ts`. One test file per source file."

The Pattern

Bad principles use adjectives: *descriptive, thorough, appropriate, organized*

Good principles use rules + examples: *numbers, patterns, file paths, tool names*

Process: Identify implicit standard → Ask "How specifically?" → Answer with rule → Add concrete example

7 Best Practices for AI-Native Testing

1. Human Oversight is Non-Negotiable

Three-layer review: **Spec review** before generation → **Test code review** after generation → **Execution review** (mutation testing). All AI-generated tests require human validation.

2. Generate from Specs, Not Implementation

❌ "Generate tests for the `calculateDiscount` function" → oracle hallucination risk

✅ Provide spec with concrete expected values → AI writes test → human validates oracle

Constitution rule: "NEVER generate tests by reading implementation code"

3. Strategic Generation, Not Exhaustive

Equivalence class partitioning: one test per class, not every variation. 47 AI-generated email tests → 8 strategic tests → same 82% mutation score.

4. Self-Healing Tests

Avoid brittle CSS selectors that break on AI refactoring. Use:

- `data-testid` attributes or natural language (Stagehand): `await page.act("click the submit button")`

5. Modular Test Architecture

❌ `beforeAll` shared setup → one bug cascades to 500 failures

✅ `beforeEach` local setup → failures isolated, parallel execution possible

Extract common setup to **stateless helper functions** (`createTestUser()` , `setupMockAPI()`)

6. Smart Test Execution with Tags

Tag tests: `@unit` , `@integration` , `@e2e` , `@fast` , `@critical` , `@smoke`

Context	Run	Time
Pre-commit	<code>@fast</code>	30 sec
PR pipeline	<code>@fast</code> + feature	5 min
Main branch	All + mutation	47 min

7. Mutation Testing — The Quality Gate

Deliberately introduce bugs (mutants); verify tests catch them.

- **Killed mutant** = test effective ✅
- **Survived mutant** = test has gap ❌
- Tools: Stryker (JS/TS), mutmut (Python), Pitest (Java)
- **Target:** ≥75% mutation score required to merge

SpecKit Workflow: Constitution → Tests

Complete Configure → Generate → Validate Cycle

Step 1: Configure (`/speckit.constitution`)

- Agent assists with all 8 sections (~30 min)
- Makes configuration feasible in one session
- Commits Testing Principles to Git

Step 2: Generate Test Tasks (`/speckit.tasks`)

Reads constitution **Sections 1–4**:

- Section 1 → test-first ordering (TDD)
- Section 2 → pyramid distribution (unit first)
- Section 3 → file locations
- Section 4 → naming conventions

→ Output: ordered test task list grouped by pyramid layer

Step 3: Implement Tests (`/speckit.implement`)

Reads constitution **Sections 5–8**:

- Section 5 → AAA anatomy
- Section 6 → mocking strategy
- Section 7 → quality criteria (blocks tautological tests)
- Section 8 → tool syntax (Jest, Playwright)

<epam> : RED phase (failing test) → GREEN phase (minimal implementation)

Example: STORY-001 User Authentication

Acceptance criteria (4) → Tests generated:

Unit tests (3): `/tests/unit/auth/`
- email validation
- password validation
- login service logic

Integration tests (2): `/tests/integration/api/`
- POST `/auth/login` endpoint
- POST `/auth/register` endpoint

E2E test (1): `/tests/e2e/flows/`
- complete login flow

Ratio: ~50/33/17, adjusted for story complexity

Without vs With Constitution

✗ **Without:** AI guesses file locations, writes tautological tests, hallucinated oracles

✓ **With:** Precise locations, quality-enforced assertions, TDD workflow

Lab: Configure Testing Principles in Constitution

60-Minute Hands-On Activity

Phase 1: Review (10 min)

- Open `/speckit.constitution` agent
- Locate Testing Principles section
- Note which sections need customization
- Consider: tech stack, risk tolerance, team experience

Phase 2: Customize (25 min)

Complete all 8 sections for YOUR project:

- ☐ Testing Philosophy (TDD/BDD, test-first?)
- ☐ Coverage Requirements (targets, exclusions)
- ☐ Test Types & Organization (folder structure)
- ☐ Naming Conventions (your pattern + example)
- ☐ Test Anatomy (AAA, beforeEach)
- ☐ Mocking & Test Data (rules by test type)
- ☐ Quality Criteria (anti-patterns, mutation score)
- ☐ Tools & Frameworks (specific versions + commands)

Phase 3: Validate with AI (15 min)

Provide a sample story to AI and ask:

"Describe your test plan — reference the testing principles from the constitution"

Check AI response:

- ☒ References constitution explicitly?
- ☒ Follows coverage distribution?
- ☒ Uses correct frameworks and file locations?
- ☒ Naming convention matches your pattern?

Note any gaps for refinement.

Phase 4: Refine (10 min)

- Find vague sections → make explicit
- Add specific examples where AI misunderstood
- Re-validate if time allows
- Commit to Git (**Required**)

Success Criteria

All 8 sections complete, standards explicit enough that AI references them precisely in validation

Module 07 Complete — Time to Build!

What You've Built

- Testing Principles configured in Speckit constitution (all 8 sections)
- Experience with the complete cycle:
 - **Configure** → **Generate** → **Validate**
- AI-native testing mindset:
 - Standards architect, not test writer
 - Human as validator, AI as generator

Key Takeaways

1. **Standards > tests:** Explicit constitution beats tribal knowledge
2. **Specs first:** Generate from specifications, validate oracles
3. **Quality gate:** Mutation testing (≥75%) is non-negotiable
4. **Avoid failure modes:** Oracle hallucination, tautological tests, duplication

Before You Go (REQUIRED)

- ✅ Save all changes (constitution with all 8 Testing Principle sections)
- ✅ Commit everything to Git
- ✅ Share repository link with instructor

Open Discussion

- What was hardest to make explicit in your constitution?
- What did the AI misunderstand first — and how did you fix it?
- What testing principles do you wish your team had documented earlier?

Next: Project Sprint — InnovatEPAM Portal

What you'll build: Employee innovation management platform from scratch

What you'll bring: PRD → Specs → ADRs → MCP → Speckit → Testing Principles — your full AI-native toolkit

Critical Prep (do in order)

1. 📖 Read [course-project/participant/speckit-guide.md](#) — internalize the workflow
2. 📄 Review [course-project/participant/00_PROJECT_OVERVIEW.md](#) — project requirements
3. 🦊 Practice Speckit workflow on 2–3 additional stories — builds muscle memory

Teams who practice before the sprint report a dramatically smoother experience